Generated PDF convenience snapshot
Authoritative source: `docs/guides/PROJECT_HANDOFF_AND_FUTURE_ROADMAP.md`
Source hand-off baseline: `9e66c00f99408c506a0195d26d8028a61563fed2`
Generated: 2026-08-23
If this PDF and the Markdown source differ, the Markdown source controls.

Project Handoff and Future Roadmap

Status

Authoritative successor-team handoff guide.

Current as of 2026-08-23.

This document describes the state of the Claude Code Backend Benchmark after:

- completion of Phases 1, 2, and 3;
- the Phase-3-compatible Kimi K3 addendum;
- comprehensive Phase 3 evidence review;
- dashboard scope/provenance correction;
- failure-taxonomy and cost-decomposition work;
- provider-billed OpenAI cost reconciliation through DR-304;
- live-supervision and canonical-publication implementation;
- completion of the Dashboard Research Guide;
- completion of the Codebase Guide.

It also defines the intended order of successor-team work.

This document does **not** activate Phase 4.

Purpose

The project is being handed to a successor team with a substantial completed research corpus and a working evidence platform.

The most important handoff objective is not:

> maintain the software exactly as it is.

It is:

> preserve enough methodological and technical understanding that the next team can discover new, defensible insights from the existing evidence and then improve the platform or design new experiments without destroying the provenance that makes those insights credible.

The repository should be treated as a research instrument.

Its primary products are:

- ideas;
- empirical findings;
- comparisons;

- anomalies;
- explanations;
- methodological lessons;
- new research questions.

Code exists to make those products easier to discover, test, reproduce, and communicate.

Read these documents first

A new team member should begin with:

```
docs/guides/DASHBOARD_RESEARCH_GUIDE.md
```

Then:

```
docs/guides/CODEBASE_GUIDE.md
```

Then this roadmap.

Use the Dashboard Research Guide to answer:

> What can we learn from the evidence already collected?

Use the Codebase Guide to answer:

> Why does the platform behave this way, and which implementation boundaries protect the research meaning?

Use this document to answer:

> What should the successor team do next?

Project research arc

The completed and planned phases answer different kinds of questions.

Phase	Principal variable	Status	Research role
Phase 1	model/backend	complete and frozen	initial Claude Code backend comparison
Phase 2	model/backend	complete and frozen	expanded direct-path backend comparison
Phase 3	provider/backend route	complete and closed	router-mediated provider expansion
Kimi K3 addendum	model/backend within Phase-3-compatible route	complete reviewed extension	extended comparison, not Phase 4
Phase 4	agent harness	planned	compare coding-agent harnesses
Phase 5	agent procedure	planned after Phase 4	explicit plan-then-execute study

This separation is methodological.

Do not collapse all future experiments into a single growing model leaderboard.

Current project state

At handoff:

- `main` is the completed post-Phase-3 baseline;
- development should use short-lived feature branches;
- Phase 1 is frozen;
- Phase 2 is frozen;
- Phase 3 is closed;
- Kimi K3 is an extended Phase-3-compatible addendum;
- the original Phase 3 reviewed core remains separately reproducible;
- the current Phase 3 extended reviewed comparison includes Kimi K3;
- comprehensive evidence review is complete;
- the J2 failure/trajectory taxonomy is frozen;
- DR-302 failure composition is implemented;
- DR-303 historical spend decomposition is implemented;
- DR-304 provider-aware current selected cost is implemented;
- historical and current cost layers remain separately visible;
- live supervision exists;
- final canonical publication exists;
- Supabase and R2 publication infrastructure exists;
- the dashboard is an evidence/research platform;
- the Planner remains review-first;
- protected dashboard dispatch is intentionally deferred;
- Phase 4 is planned but not activated;
- Phase 5 remains after Phase 4.

Current reviewed Phase 3 populations

Core

The original Phase 3 reviewed full-suite population:

```
15 arms
900 trials
515 raw successes
```

This is the population behind the original Phase 3 closeout comparison.

Extended

The current reviewed Phase 3 comparison:

```
16 arms
960 trials
562 raw successes
```

This adds Kimi K3.

The extended population is normally the preferred current reviewed Phase 3 comparison when all required metrics are supported.

Dynamic operational evidence

The database also contains dynamic imported evidence.

That population can include:

- canary;
- smoke;
- diagnostics;
- multiple runs;
- invalid/quarantined runs in broader inventory views.

Do not treat dynamic imported inventory as interchangeable with the frozen reviewed 960-trial population.

Frozen versus current versus dynamic

Frozen historical evidence

Examples:

- Phase 1 aggregate/results;
- Phase 2 aggregate/results;
- accepted Phase 3 scored corpus;
- original reviewed Phase 3 comparison;
- comprehensive review snapshot;
- J2 taxonomy snapshot;
- DR-302 source semantics;
- DR-303 historical spend semantics;
- historical reports.

These should remain reproducible.

Current reviewed interpretation

Examples:

- DR-304 current selected-cost layer;
- current reviewed Phase 3 extended comparison;
- exact reviewed run-selection contract.

A new current layer may supersede an older decision-facing interpretation without rewriting the historical evidence.

Dynamic operational state

Examples:

- all imported runs;
- valid imported runs;
- current database inventory;

- live runs;
- current artifact availability.

This can change over time.

It should be labeled accordingly.

First principle for the successor team

Do not begin with a refactor.

Begin by finding research insights.

The completed dashboard and evidence corpus are sufficiently rich that there is substantial analytical work available before another benchmark needs to run.

The recommended sequence is:

```
use dashboard
  -> discover pattern
  -> formulate hypothesis
  -> trace evidence
  -> inspect counterexamples
  -> read relevant implementation
  -> determine limitation
  -> decide whether analysis, software, or a new experiment is needed
```

This order protects the project from becoming an engineering exercise with no research question.

Why dashboard work comes before code changes

A successor engineer may immediately notice:

- duplicate-looking reporting layers;
- historical/current cost fields;
- multiple database views;
- generated snapshot modules;
- overlapping failure classifications;
- live and canonical tables;
- exact-run and latest-run concepts.

Many of these are intentional provenance boundaries.

Using the dashboard first gives those boundaries meaning.

For example:

- current and historical cost layers exist because later provider billing corrected historical benchmark telemetry;
- exact reviewed run selection exists because reproducibility is more important than silently showing the newest run;
- live and canonical tables differ because partial mutable execution state is not final benchmark truth;
- comprehensive review and J2 differ because they answer different behavioral questions.

Understanding the research problem prevents destructive simplification.

Successor-team first activity: insight discovery

The team's first substantial work product should be an **insight memo**, not a code change.

Use:

```
docs/guides/DASHBOARD_RESEARCH_GUIDE.md
```

The memo should contain:

- several reproducible observations;
- supporting evidence paths;
- at least one counterexample per strong claim where practical;
- evidence limitations;
- alternative explanations;
- follow-up questions;
- candidate experiments only where the current corpus cannot answer the question.

Good research directions include:

- cost/performance niches;
- task-specific model specialization;
- concentrated task-family weaknesses;
- timeout-heavy versus verifier-heavy failure profiles;
- policy-refusal behavior;
- response-path reliability;
- successful but operationally unclean runs;
- near-miss patterns;
- provider-family differences;
- direct/router-associated differences;
- evidence-completeness limitations;
- cost-accounting effects on model choice.

Second activity: codebase comprehension

After initial dashboard research, the team should trace important findings into implementation.

Use:

```
docs/guides/CODEBASE_GUIDE.md
```

The goal is not memorization.

The goal is to answer:

- Which population produced the number?
- Which evidence source is authoritative?
- Which layer is reviewed?
- Which layer is live?
- Which fields are historical?

- Which fallback behaviors exist?
- Which tests enforce the contract?
- What would change if this code were modified?

This targeted code reading builds confidence faster than broad repository refactoring.

Research output should drive engineering

A useful engineering request looks like:

> We cannot answer whether timeout-heavy failures cluster by task family without manually joining two reviewed views; add a tested dashboard view that preserves the frozen population and makes that comparison explicit.

A weak engineering request looks like:

> This code feels complicated; combine the data models.

Prefer the first kind.

Sponsor-facing priority

The sponsor-facing value of the project is primarily:

- genuine findings;
- surprising comparisons;
- useful explanations;
- research concepts;
- actionable methodological lessons.

A large software change with no new insight is a supporting deliverable.

A small software change that exposes an important finding can be highly valuable.

The successor team should therefore track:

```
research value unlocked
```

alongside:

```
engineering work completed
```

Operational infrastructure

Six-runner topology

The current six-slot self-hosted runner topology remains operationally relevant.

Logical slot labels:

```
cc-bench-slot-1
cc-bench-slot-2
cc-bench-slot-3
```

```
cc-bench-slot-4
cc-bench-slot-5
cc-bench-slot-6
```

The currently documented arrangement distributes those slots across two VPS hosts.

Use slot and pool labels for scheduling.

Do not parse runner display names for routing logic.

Infrastructure horizon

The current VPS arrangement is funded and expected to remain available through **December 2026**.

There is no need to redesign the runner fleet solely because ownership is changing.

Before that infrastructure horizon is reached, the successor team should decide whether to:

- renew the existing VPS arrangement;
- migrate to replacement hosted runners;
- move some work to local/on-premise systems;
- use another cloud provider;
- combine permanent and burst capacity;
- redesign the runner pool for future phases.

That decision should be driven by the Phase 4/5 workload and operating cost, not made prematurely.

Infrastructure transition objective

By the time the present VPS arrangement approaches expiration, the successor team should know:

1. expected future benchmark volume;
2. whether Phase 4 requires different runner software;
3. whether additional storage/network capacity is needed;
4. whether harness-specific dependencies change runner images;
5. whether runner-slot concurrency remains useful;
6. what monthly idle cost is acceptable;
7. whether self-hosting remains worth the operational burden.

The transition should preserve reproducibility and runner provenance.

What not to do now

Do not redesign infrastructure solely to make the handoff look cleaner.

The existing fleet already supports useful future work.

Use it while learning what the next experimental phase actually requires.

Live supervision

Live supervision is an implemented capability.

It is not an unfinished idea that must be rewritten before future benchmark work.

Its current purpose is to make execution observable while preserving the boundary:

```
provisional live state
  !=
final canonical benchmark evidence
```

Current live flow

Conceptually:

```
runner
  -> Harbor / agent execution
  -> redacted local live evidence
  -> optional live Supabase metadata
  -> optional progressive R2 artifacts
  -> completed Harbor result
  -> final canonical publisher
  -> verified R2 evidence
  -> canonical Supabase metadata
```

The local dashboard reads shared services.

It does not require a direct inbound connection to the VPS hosts.

Live database state

Live tables include:

```
benchmark.live_runs
benchmark.live_run_events
benchmark.live_trials
benchmark.live_artifacts
```

They are mutable execution-observation state.

A live run can later link to a canonical arm run after final publication.

Do not treat live rows as an alternative canonical corpus.

Canonical publication

The final publisher:

- discovers the execution-specific result directory;
- verifies structural completion;
- builds the canonical manifest;
- verifies local artifacts;
- reconciles progressive evidence;
- verifies R2;
- inserts canonical database state transactionally;
- verifies canonical relationships;
- links supervised live state to canonical state;
- protects replay identity.

This is already a substantial reliability layer.

Future changes should extend it rather than bypass it.

Migration 009 status

Migration:

```
db/migrations/phase3/009_live_run_supervision.sql
```

was already applied historically.

Do not rerun it.

Some older live-supervision documentation still contains a deployment sequence written before permanent application.

Treat those instructions as historical implementation provenance.

Current-state documentation should not instruct a new operator to apply migration 009 again.

The synchronized live-supervision runbook now makes that distinction explicit.

Closed Phase 3 publication

Completed Phase 3 suites remain closed to ordinary new canonical publication.

Repair publication requires explicit authorization.

Do not use repair authorization as a convenient way to add new experiments to Phase 3.

Use a future-phase identity.

Reasonable future live-supervision improvements

Possible successor-team improvements include:

- clearer live/canonical reconciliation UX;
- better run-progress summarization;
- more explicit stuck/stale-run diagnostics;
- improved artifact retrieval health;
- clearer warnings when progressive evidence is incomplete;
- better retry/recovery operator guidance;
- future-phase generalization of Phase-3-named scripts;
- stronger runner-health visibility;
- improved publication-failure triage;
- richer linking between GitHub Actions, live run, canonical run, and reviewed comparison.

These should be prioritized only when they solve a real operating or research problem.

What live supervision should not become

Do not turn live supervision into:

- a second scoring system;
- an alternate canonical store;
- a hidden-reasoning viewer;
- an unbounded process-log database;
- a mechanism that changes benchmark success because telemetry failed.

Observation must remain failure-isolated from benchmark execution.

Dashboard Planner

The Planner is presently **review-first and non-dispatching**.

It can help construct and validate intended work.

It should not be described as a live execution controller.

This boundary is intentional.

Protected dashboard dispatch is deferred

Protected server-side dashboard dispatch remains reasonable future platform work.

It is **not** part of the current handoff implementation.

It is also **not** a prerequisite for Phase 4.

The successor team may implement it if dashboard-driven execution becomes valuable after familiarization with the platform.

Intended protected-dispatch architecture

A future design should include:

Server-side credentials only

GitHub credentials must remain on the server.

No workflow token belongs in browser JavaScript.

Allowlisted execution

Only known:

- arms;
- phases;
- modes;
- task sets;
- runner labels

should be dispatchable.

Dry-run-first behavior

The default action should be a zero-cost/dry execution path.

Paid execution should require stronger intent.

Explicit paid-run authorization

A protected UI must preserve the workflow's explicit paid-run confirmation semantics.

Convenience must not silently remove the human cost gate.

Runner capacity awareness

The dispatcher should distinguish:

- number of arm jobs;
- runner slots;
- per-arm Harbor concurrency;
- resulting maximum task concurrency.

Provider-family capacity awareness

Runner availability and provider quota are different resources.

A free runner slot does not mean another simultaneous provider request stream is safe.

Provider-family limits should be configurable evidence-backed policy, not hardcoded forever from one historical smoke run.

Audit record

Every dispatch should record:

- requester;
- requested phase/arm/mode;
- dry/paid state;
- runner target;
- concurrency;
- timestamp;
- resulting GitHub workflow execution;
- later publication/canonical result.

Workflow linkage

The dashboard should link:

```
plan
  -> dispatch
  -> GitHub Actions run
  -> live execution
  -> canonical run
  -> reviewed result
```

when each stage exists.

No silent bypass

Protected dispatch must not bypass:

- workflow validation;
- paid-run confirmation;
- closed-suite publication guard;
- secret separation;
- canonical publication checks.

Why protected dispatch is not urgent

The current review-first Planner plus GitHub Actions dispatch already provides a usable operating path.

Server-side dashboard dispatch would improve convenience.

It does not currently unlock a major research question.

Therefore it should remain behind higher-value work such as:

- corpus analysis;
- insight discovery;
- Phase 4 methodology preparation.

Dashboard future work

Future dashboard changes should be prioritized by research questions.

High-value examples might include:

- easier task-family comparison;
- richer failure-composition cross-filtering;
- confidence/evidence-completeness comparison;
- direct support for hypothesis notebooks or saved filters;
- clearer matched-task cross-phase comparisons;
- aggregate-to-trial research packets;
- Phase 4 harness-aware comparisons;
- future Phase 5 planning/execution decomposition.

Do not add visualizations merely because another chart fits.

Ask what decision or research question it improves.

Phase 4

Phase 4 is the next planned benchmark phase.

It should begin **after** the successor team has completed initial handoff orientation and existing-corpus research.

This handoff does not activate Phase 4.

Phase 4 research question

Phase 4 asks:

> How do different coding-agent harnesses perform on the same Terminal-Bench tasks when model/provider choices are controlled as much as practical?

This changes the system under test from:

```
Claude Code fixed
  + varying backend/provider
```

to:

```
varying agent harness
  + comparable backend/provider choices
```

That makes Phase 4 methodologically separate from Phases 1–3.

Why Phase 4 matters

Phases 1–3 establish that model/backend choice matters under a fixed principal harness.

They do not establish how much observed performance comes from:

- model capability;
- Claude Code's tool loop;
- context management;
- editing behavior;
- retry behavior;
- planning;
- permissions;
- shell strategy;
- harness-specific prompting.

Phase 4 begins to separate those effects.

Existing Phase 4 candidate harnesses

The existing plan identifies candidates such as:

- Claude Code as control;
- Codex / OpenAI-native coding agent;
- Gemini CLI / Gemini-native coding agent;
- OpenHands / SWE-agent-style harness;
- custom Harbor agent;
- other practical Harbor-supported coding agents.

This list should be revalidated at activation time.

Agent products and Harbor integrations evolve.

Do not assume the 2026-08 candidate list is permanently correct.

Phase 4 activation criteria

Before the first paid Phase 4 canary, the successor team should complete the following.

Research orientation

- reproduce several existing dashboard findings;
- understand core/extended/imported scopes;
- understand reviewed versus canonical versus live evidence;
- understand current versus historical cost.

Code orientation

- understand current Harbor invocation;
- identify Claude-Code-specific assumptions;
- understand artifact ingestion;
- understand publication fingerprint;
- understand closed Phase 3 safety.

Harness selection

- choose a small initial candidate set;
- document why each harness is scientifically interesting;
- identify a meaningful control;
- avoid adding arms solely because they are available.

Compatibility matrix

For each candidate harness record:

- installation;
- Harbor support;
- adapter needed;
- model support;
- tool interface;
- file-edit strategy;
- shell behavior;
- permission model;
- transcript format;
- trajectory availability;
- token availability;
- cost availability;
- version capture;
- exit semantics;
- timeout semantics.

Evidence normalization

Define a common minimum evidence contract.

Do not require every harness to imitate Claude Code's artifact format exactly.

Require enough normalized evidence to compare behavior honestly.

Metric normalization

Define common:

- success/reward;
- total runtime;
- agent runtime where observable;
- cost;
- token accounting state;
- exception state;
- tool/action counts where comparable.

Mark unavailable metrics unavailable.

Do not invent zeroes.

Contamination policy

Determine whether each harness can satisfy the benchmark's tool restrictions.

If a harness exposes a fundamentally different capability set, document it as part of the experimental condition.

Budget

Estimate canary and smoke cost before any full sweep.

The smaller, cleaner experiment is preferable to a broad inconsistent one.

Phase 4 implementation sequence

Stage P4-A — Refresh the plan

Review:

```
docs/plans/phase4/PHASE4_AGENT_HARNESS_PLAN.md
```

Update candidate availability and current Harbor capabilities.

Do not erase the original planning history.

Stage P4-B — Build compatibility matrix

Produce a checked-in research/design artifact covering the candidate harnesses.

No paid runs are required.

Stage P4-C — Generalize only what is necessary

Audit existing Claude-Code-specific assumptions.

Potential adaptation points include:

- config model;
- Harbor agent selection;
- transcript artifact naming;
- trajectory normalization;
- runtime-version capture;
- artifact registry;
- dashboard presentation.

Avoid a giant generic-agent framework before the first canary demonstrates a need.

Stage P4-D — Zero-cost/local integration check where possible

Prefer a zero-cost local integration check when the harness supports one.

Verify:

- harness starts;
- adapter loads;
- command construction is correct;
- secrets remain isolated;
- artifacts are captured.

If meaningful validation inherently requires a provider call, document that limitation and make the one-task canary the first minimal paid check rather than inventing a fake local equivalence.

Stage P4-E — Canary

Run one Terminal-Bench task per candidate harness.

Existing suggested canary:

```
modernize-scientific-stack
```

Confirm:

- task execution;
- verifier;
- artifacts;
- scoring;
- cost/accounting state;
- no secret leak.

Stage P4-F — Five-task smoke

Use:

```
configs/tasks/phase2-smoke.txt
```

Acceptance should focus on infrastructure stability rather than winning the benchmark.

Stage P4-G — Review before full sweep

Compare:

- launch reliability;
- artifact completeness;
- verifier compatibility;
- metric comparability;
- cost;
- qualitative harness behavior.

Remove or fix unstable arms before full execution.

Stage P4-H — Full scored experiment

Only after acceptance:

```
20 tasks
  x 3 attempts
  x selected harness/model arms
```

Keep the arm set budget-controlled.

Stage P4-I — Qualitative evidence review

Do not stop at aggregate pass rate.

Compare:

- tool use;
- failure mechanisms;
- retries;

- edit behavior;
- task-family patterns;
- timeout behavior;
- evidence completeness.

Stage P4-J — Report

Produce:

- aggregate;
- harness compatibility findings;
- failure analysis;
- qualitative comparison;
- cost comparison;
- methodological limitations;
- new hypotheses.

Phase 4 output isolation

Use:

```
results/phase4/
figures/phase4/
docs/reports/phase4/
```

Do not put Phase 4 scored results under `results/phase3/`.

Phase 4 control design

The Claude Code control is valuable because Phases 1–3 provide substantial historical evidence for that harness.

But historical results are not automatically perfect contemporaneous controls.

At Phase 4 activation, decide whether a fresh Claude Code control is required to account for:

- harness-version drift;
- provider/model revisions;
- environment changes;
- Harbor changes;
- time effects.

Document that choice.

Phase 4 confounds

A native harness comparison can conflate:

- model;
- provider;
- harness;
- tool schema;
- system prompt;
- context strategy;

- editing method;
- retry policy;
- permission model.

Do not report Phase 4 as a pure model ranking.

The object of study is the combined harness condition.

Phase 4 dashboard implications

Future dashboard work may need a new first-class dimension:

```
agent harness
```

Do not encode harness identity only into an arm display string.

Likely needs include:

- harness field;
- harness version;
- normalized evidence completeness;
- harness-specific artifact presentation;
- cross-harness comparison;
- task × harness views;
- harness-aware failure categories.

Design these only after the initial compatibility work shows what is actually needed.

Phase 4 stop conditions

Stop before full sweep if:

- harness repeatedly fails to launch;
- verifier integration is unreliable;
- task environment is no longer comparable;
- artifacts are insufficient for interpretation;
- secrets can leak;
- cost cannot be bounded;
- the chosen configuration changes too many variables to answer the intended question.

A documented blocker is better than invalid benchmark data.

Phase 5

Phase 5 remains **after Phase 4**.

It changes another dimension:

```
agent procedure
```

rather than merely:

```
backend
```

or:

```
harness
```

Phase 5 research question

The existing plan asks whether explicitly incorporating planning improves:

- success;
- cost;
- latency;
- failure behavior.

Its core proposed structure is:

```
planning pass
  -> saved plan
  -> execution pass
  -> normal verifier
```

Why Phase 5 stays after Phase 4

Phase 4 teaches the project how to compare different harness execution models.

That experience should make Phase 5 stronger.

If Phase 5 were started first, changes could be conflated across:

- model;
- harness;
- planning procedure.

Keeping the sequence:

```
backend comparison
  -> harness comparison
  -> procedure comparison
```

makes interpretation cleaner.

Initial Phase 5 target

The existing plan recommends starting with the simple auto-accept one-shot variant.

Planning pass:

- inspect task/repository;
- make no persistent solution edits;
- save `plan.md`;
- retain planner evidence.

Execution pass:

- receive original task plus plan;
- edit the actual task environment;
- retain executor evidence;
- run normal verifier.

Existing recommended Phase 5 smoke arms

The current plan suggests an initial smoke involving:

```
planexecute-sonnet-sonnet
planexecute-opus-sonnet
planexecute-deepseek-pro-flash
```

using the five-task smoke list.

That candidate set should be revalidated after Phase 4.

Phase 4 findings may change which harness/model combinations make the most scientific sense.

Phase 5 evidence requirements

Phase 5 introduces new evidence such as:

- plan artifact;
- planner transcript;
- planner trajectory;
- planner runtime;
- planner tokens;
- planner cost;
- executor transcript;
- executor trajectory;
- executor runtime;
- executor tokens;
- executor cost.

The system should make planning and execution costs separable.

Do not hide two-pass cost inside one opaque total.

Phase 5 analytical questions

Useful questions include:

- Did planning improve raw success?
- Did it improve clean success?
- Did it reduce unproductive iteration?
- Did it reduce near misses?
- Did it increase latency too much?
- Did planning cost more than the success gain justified?
- Did executors follow good plans?
- Did bad plans cause otherwise avoidable failure?
- Are planning benefits task-family specific?

Phase 5 output isolation

Use:

```
results/phase5/
figures/phase5/
docs/reports/phase5/
```

Do not mix Phase 5 plan-execute trials into single-pass Phase 2/3 aggregates.

Safe benchmark-change workflow

Future benchmark changes should follow a predictable sequence.

1. State the research question

Write down exactly what variable is intended to change.

2. Identify the control

What condition makes the change interpretable?

3. Identify confounds

What else changes unintentionally?

4. Define evidence requirements

Which artifacts and metadata are required to interpret the result?

5. Define cost boundary

What is the maximum acceptable exploratory and full-run spend?

6. Define canary

Use the cheapest realistic proof of integration.

7. Define smoke

Use a small heterogeneous task subset.

8. Review smoke evidence

Do not judge only by pass rate.

Inspect artifacts.

9. Decide whether full sweep is justified

The decision should be explicit.

10. Publish under a new phase/suite identity

Do not reopen a frozen phase.

11. Review and report

Preserve raw evidence and add interpretations rather than rewriting the past.

Before every paid run

At minimum confirm:

- intended branch/commit;
- arm config;
- task set;
- attempts;
- concurrency;
- runner target;
- provider availability;
- provider quota;
- cost estimate;
- contamination controls;
- secret configuration;
- live/publication controls;
- output path;
- phase/suite identity.

Before every publication change

Confirm:

- publication eligibility semantics;
- fingerprint semantics;
- R2 verification;
- transaction behavior;
- replay behavior;
- closed-suite policy;
- migration requirements;
- rollback tests.

Publication changes deserve greater caution than presentation changes.

Before every reviewed-evidence change

Confirm:

- raw evidence remains unchanged;
- exact population is defined;
- generator version is explicit;
- source hashes are pinned;
- output manifest is complete;
- old reviewed snapshot remains preserved;
- dashboard can distinguish old and new layer.

Before every dashboard metric change

Confirm:

- source population;
- authority;
- denominator;
- missing-value semantics;
- provenance link;
- confidence/qualification;
- whether the metric is current or historical.

Successor-team first week

The first week should focus on reproduction, research, and orientation.

Day 1 — Project and dashboard orientation

Read:

```
docs/guides/DASHBOARD_RESEARCH_GUIDE.md
docs/guides/CODEBASE_GUIDE.md
docs/guides/PROJECT_HANDOFF_AND_FUTURE_ROADMAP.md
```

Explore:

```
Overview
Architecture
Data Model
Glossary
```

Be able to explain:

- Phase 3 core;
- Phase 3 extended;
- valid imported;
- all imported;
- reviewed versus live;
- current versus historical cost.

Day 2 — Reproduce headline economics

Use:

```
Overview
Cost Coverage
Cross-phase
```

Reproduce:

- Phase 3 reviewed population;
- selected current cost;

- historical cost;
- OpenAI provider-billed correction;
- Kimi K3 qualification.

Do not modify code.

Day 3 — Failure mechanisms

Use:

```
Trial Quality
Evidence Review
trial detail
artifact detail
```

Pick two arms with similar pass rates and compare how they fail.

Record representative and contradictory evidence.

Day 4 — Task behavior

Use:

```
Eval Suites
Evals
task heatmaps
trial drilldown
```

Identify:

- one broadly difficult task;
- one model-specific weakness;
- one counterexample to an apparent model-family pattern.

Day 5 — Code trace

Choose one finding from Days 2–4.

Trace:

```
dashboard number
  -> data/model layer
  -> reviewed/operational source
  -> exact run
  -> exact trial
  -> artifact evidence
```

Read only the code needed to explain that finding.

First-week deliverable

Produce a short research memo with:

- at least three findings;
- evidence links;

- uncertainty;
- counterexamples;
- three unresolved research questions;
- candidate dashboard improvements;
- candidate experiment ideas.

Do not make a Phase 4 run the deliverable.

Successor-team first month

Week 1 — Reproduce and understand

Complete the first-week program.

Week 2 — Independent analysis

Investigate new questions not simply copied from existing reports.

Potential themes:

- task specialization;
- failure mechanism;
- cost-quality frontier;
- provider behavior;
- route-associated behavior;
- evidence completeness.

Week 3 — Challenge findings

For the strongest findings:

- inspect counterexamples;
- compare populations;
- inspect incomplete evidence;
- review relevant tests/code;
- distinguish association from cause.

Week 4 — Prioritize next work

Create a backlog divided into:

1. findings already supportable from current data;
2. analyses needing no platform change;
3. dashboard improvements that unlock high-value analysis;
4. operational improvements;
5. new experiments;
6. Phase 4 prerequisites.

The backlog should state research value, not only implementation effort.

First-month deliverable

A useful first-month package would contain:

- research memo;
- evidence-backed findings;
- research-question backlog;
- dashboard improvement shortlist;
- Phase 4 readiness assessment;
- infrastructure horizon check;
- no unnecessary frozen-history changes.

Suggested insight record format

For each research finding, record:

Question

What was being investigated?

Population

Which exact scope?

Observation

What does the data show?

Evidence

Which run/trial/artifact supports it?

Counterevidence

What does not fit the pattern?

Interpretation

What explanation is supported?

Qualification

What remains unknown?

Engineering implication

Does anything need to change in the platform?

Experimental implication

Does a new benchmark experiment become useful?

This format keeps ideas ahead of code.

Suggested engineering proposal format

Before a material platform change, record:

Research/operational need

Why is this valuable?

Current limitation

What cannot be answered or operated safely today?

Proposed change

Which layer changes?

Protected boundaries

What must remain identical?

New evidence semantics

Does the change create a new metric, diagnosis, identity, or source?

Validation

Which tests prove the contract?

Documentation

Which current guide must change?

Common takeover mistakes

Common takeover mistake: treating current code as the research subject

The project is not primarily a study of its own software architecture.

Do not let:

- framework preferences;
- refactoring style;
- frontend fashion;
- abstraction preferences

replace benchmark research.

Improve the code when it improves:

- correctness;
- provenance;
- usability;
- reliability;
- research throughput;
- insight discovery.

Common takeover mistake: reopening Phase 3

Phase 3 is complete.

Do not add a new full sweep to Phase 3 merely because:

- the router config already exists;
- the workflow already exists;
- the publisher already knows Phase 3;
- the dashboard already displays Phase 3.

Use a future phase.

Common takeover mistake: overwriting a reviewed snapshot

If a newer interpretation becomes preferable:

```
preserve old snapshot
  -> add new evidence
  -> generate new reviewed layer
  -> document supersession
```

Do not:

```
edit old snapshot until it says the new thing
```

Common takeover mistake: treating current cost as trial allocation

Provider-billed OpenAI arm totals are current authoritative arm-level totals.

They do not provide provider-billed per-trial or per-outcome allocation.

Do not manufacture one.

Common takeover mistake: assuming unavailable means zero

This applies to:

- cost;
- tokens;
- artifacts;
- taxonomy;
- router evidence;
- provider logs.

Unavailable is evidence state.

Zero is observed value.

Common takeover mistake: treating live as final

Live state can be:

- partial;
- mutable;
- stale;

- missing;
- recovered.

Canonical publication is the stronger final operational layer.

Reviewed snapshots can then create an even more controlled analytical layer.

Common takeover mistake: treating successful as clean

A raw success can still carry:

- timeout evidence;
- telemetry anomalies;
- questionable execution validity;
- other behavioral qualifications.

Use independent axes.

Common takeover mistake: treating failure as wrong code

A failure may represent:

- verifier/task failure;
- timeout;
- provider-policy refusal;
- invalid response path;
- missing output;
- extraneous artifacts;
- incomplete evidence.

Use the accepted taxonomy rather than one generic failure label.

Common takeover mistake: treating router association as router causation

Cross-phase direct-versus-router comparisons include confounds.

Use them to generate controlled hypotheses.

Do not automatically attribute every delta to LiteLLM.

Common takeover mistake: assuming current runtime proves historical runtime

A version installed today is not proof of the version used for an older run.

Use retained runtime provenance when available.

Common takeover mistake: treating Planner assumptions as live capacity

Planner validation rules are planning policy.

They are not guaranteed live provider quota or runner health.

Revalidate capacity before paid work.

Current documentation caveat

Several older project files were written while Phase 3 was still active.

Their historical bodies should remain preserved.

Current-state readers should prefer:

```
docs/guides/
```

plus later closeout/review documents.

The handoff synchronization work added explicit historical/superseded status notices where necessary.

Infrastructure risk register

VPS horizon

Current arrangement available through December 2026.

Mitigation:

- evaluate future benchmark volume before renewal;
- select replacement before expiration;
- avoid last-minute migration during a paid sweep.

Provider access drift

Provider model access can change.

Mitigation:

- cheap availability check before experiment;
- pin observed model evidence;
- do not assume historical slug remains current.

Harness drift

CLI agents evolve rapidly.

Mitigation:

- capture exact harness/runtime version;
- consider contemporaneous controls.

Storage drift

R2/database configuration can change.

Mitigation:

- keep publication verification;
- preserve artifact hashes;
- test read paths before major run.

Review-model drift

Offline classifiers/generators can evolve.

Mitigation:

- version generators;
- bind source hashes;
- add new reviewed layers.

Documentation drift

Historical plans can look current.

Mitigation:

- status banners;
- authoritative guides;
- do not rewrite historical bodies.

Research-validity risk register

Cross-phase time effects

Older and newer runs occurred at different times.

Do not interpret all differences as model or route effects.

Harness/model confounding in Phase 4

Native agent harnesses may require their own models.

Report the combined experimental condition.

Cost incomparability

Different providers expose different billing granularity.

Preserve confidence and allocation state.

Artifact incomparability

Different harnesses may not produce identical transcripts.

Define common evidence concepts rather than forcing fake equivalence.

Tool-capability differences

Harnesses may expose different tools.

Treat capability differences as experimental condition.

Budget-driven sample size

Do not overstate a small smoke study.

Keep canary/smoke/full labels explicit.

Sponsor-facing reporting rules

Sponsor-facing material should make clear:

- exact population;
- quality metric;
- cost basis;
- routing/harness condition;
- evidence limitations;
- whether a claim is associative or causal;
- whether an observation is historical or current.

Prefer:

> In the reviewed Phase 3 extended population...

over:

> The benchmark proves...

Prefer:

> Provider billing changed the economic interpretation...

over:

> The model suddenly became cheaper...

when the benchmark run itself did not change.

Current source-of-truth starting points

Phase 1:

```
results/phase1/combined.csv
```

Phase 2:

```
results/phase2/combined.csv
```

Phase 3 closeout:

```
docs/reports/phase3/PHASE3_CLOSEOUT_INDEX_20260714.md
```

Current reviewed Phase 3:

```
results/phase3/reporting/phase3_current_reviewed_comparison_20260821.json
```

Historical reviewed Phase 3:

```
results/phase3/reporting/phase3_extended_reviewed_comparison_20260805.json
```

Reviewed run selection:

```
results/phase3/reporting/phase3_reviewed_run_selection_20260809.json
```

Comprehensive evidence:

```
results/manual_verification/comprehensive_review_20260731/
```

Failure taxonomy:

```
results/manual_verification/failure_taxonomy_20260813/
```

Operational starting points

Benchmark workflow:

```
.github/workflows/phase3-arm-dispatch-v2.yml
```

Execution:

```
scripts/run_arm.sh
scripts/lib/harbor.py
```

Live supervision:

```
scripts/run_arm_live.py
docs/runbooks/LIVE_RUN_SUPERVISION.md
```

Final publication:

```
scripts/publish_phase3_run.py
```

Dashboard:

```
apps/dashboard/
```

Database migrations:

```
db/migrations/phase3/
```

Future-plan starting points

Phase 4:

```
docs/plans/phase4/PHASE4_AGENT_HARNESS_PLAN.md
```

Phase 5:

```
docs/plans/phase5/PHASE5_PLAN_EXECUTE_PLAN.md
```

Protected dashboard dispatch history:

```
docs/plans/phase3/PHASE3_DASHBOARD_DISPATCH_AND_COST_RECONCILIATION.md
```

Immediate successor backlog

Recommended order:

Priority 1 — Research the existing corpus

Use the dashboard and evidence.

Produce findings.

Priority 2 — Understand the implementation behind important findings

Use targeted code reading.

Priority 3 — Fix only research-blocking or operationally important issues

Do not perform broad cleanup for its own sake.

Priority 4 — Improve high-value dashboard research workflows

Let observed research friction determine features.

Priority 5 — Refresh Phase 4 design

Revalidate harness candidates and compatibility.

Priority 6 — Run Phase 4 canary/smoke

Only after methodology and evidence capture are ready.

Priority 7 — Run Phase 4 scored sweep

Only after canary/smoke acceptance and budget review.

Priority 8 — Analyze and close Phase 4

Do not rush directly into Phase 5.

Priority 9 — Refresh and begin Phase 5

Use lessons from Phase 4.

Deferred platform backlog

Reasonable deferred items include:

- protected dashboard dispatch;
- live-supervision UX improvements;
- future-phase-neutral naming/generalization;
- richer runner health;
- improved saved research views;

- automated research packets;
- better cross-phase matched-task exploration;
- harness-aware dashboard dimensions;
- plan/execution-aware Phase 5 views.

These are options.

They are not prerequisites merely because they are listed.

Decision rule for future work

For each candidate task, ask:

> What new evidence, insight, reliability, or research capability does this produce?

If the answer is unclear, lower its priority.

Decision rule for new experiments

Before paying for another benchmark run, ask:

> Can the existing corpus already answer this question?

If yes, analyze first.

If no, ask:

> What is the smallest experiment that isolates the missing information?

That becomes the canary/smoke design.

Decision rule for refactoring

Before restructuring working provenance-sensitive code, ask:

> Which invariant is currently protected by this structure?

Then:

> Can the same invariant be proved by tests after the refactor?

If not, do not simplify yet.

Handoff success criteria

The handoff is successful when a successor team can independently answer:

1. What did Phases 1–3 test?
2. Why is Kimi K3 not Phase 4?
3. Which evidence is frozen?
4. Which current reviewed layer should be used for current Phase 3 cost?
5. Why do historical cost fields still exist?
6. Why can provider-billed arm totals not be allocated to individual trials?
7. What is the difference between reviewed, canonical, operational, and live?

8. How does a Harbor trial become dashboard evidence?
9. Why are live tables non-canonical?
10. Why is exact reviewed run selection important?
11. Why must migration 009 not be rerun?
12. Why is protected dashboard dispatch deferred?
13. What runner capacity exists today?
14. What is the December 2026 infrastructure horizon?
15. What should the team investigate before editing code?
16. What makes Phase 4 methodologically different?
17. What must pass before a Phase 4 full sweep?
18. Why does Phase 5 remain after Phase 4?
19. How should future reviewed interpretations preserve old evidence?
20. What should the team produce in its first week and first month?

Definition of a good successor state

A good successor state is not one where every old file has been modernized.

It is one where the team:

- trusts the evidence;
- understands its limits;
- can reproduce important findings;
- can challenge those findings;
- knows where the implementation enforces research semantics;
- knows what infrastructure is available;
- knows what is frozen;
- can safely design the next experiment;
- produces useful new ideas.

Final handoff principle

The project should continue to move in this order:

```
evidence
  -> insight
  -> question
  -> targeted engineering
  -> controlled experiment
  -> new evidence
```

not:

```
engineering
  -> more engineering
  -> benchmark because infrastructure exists
```

The platform is successful when it helps the successor team discover something important that was not obvious from a leaderboard.